Math for Data Science - Lab 9

Principal Components Analysis (PCA): An Application to Image Compression

This week we will explore the dimension reduction technique of **Principal Components Analysis** (PCA), which is a way of efficiently summarizing and describing large datasets. PCA has a large variety of applications in genomics, facial recognition and computer vision, finance and economics, climate science, and social science, but we will start by using it to represent an image of Laszlo the cat.

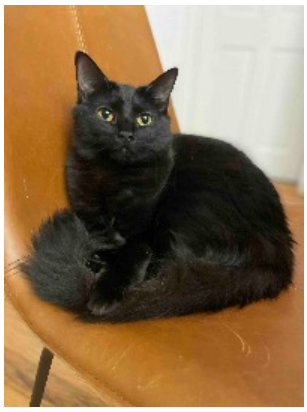


Figure 1: Laszlo the cat.

You have already learned everything you need to know to understand this powerful tool: maximization, variance, and eigendecomposition. Let's put these pieces together to see how it works.

Exercises 1-8 are the core of the lab; Exercise 9 is optional.

Setup

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

Using R instead of Python (optional)

The lab is written in Python, but you can write your answers in R inside this same notebook. This needs R 4.5 or newer, and Jupyter started with rpy2 added: `uv run --with rpy2 jupyter lab`. The `README.md` of the course folder has the details and what to do when it does not work.

If you want to use R instead of Python, uncomment and run the following cell:

```
# %load_ext rpy2.ipython
```

Then, in any subsequent code cell, add `%%R` as the first line to run that cell as R:

```
%%R
x <- rbinom(1000, size = 10, prob = 0.3)
hist(x)
```

Keep running the given Python cells (imports, data loading) as they are. The table below lists the R equivalents of the Python functions this lab uses.

Loading the image in R needs the `jpeg` package. Install it once, then load it at the top of your first R cell:

```
%%R
install.packages("jpeg")   # once
library(jpeg)
```

Python and R functions in this lab

Task	Python	R
Load the JPEG as values in $[0, 1]$	`np.asarray(Image.open("laszlo.jpg"), dtype=float) / 255`	`readJPEG("laszlo.jpg")`
Show an image	`plt.imshow(img)`	`plot(as.raster(img))`
Red channel of an $h \times w \times 3$ array	`img[:, :, 0]`	`img[, , 1]`
Column means	`X.mean(axis=0)`	`colMeans(X)`
Center the columns	`X - X.mean(axis=0)`	`scale(X, center = TRUE, scale = FALSE)`
Covariance matrix, dividing by n	`np.cov(X, rowvar=False, bias=True)`	`cov(X) * (n - 1) / n`
Variance, dividing by n	`x.var()`	`var(x) * (n - 1) / n`
Correlation	`np.corrcoef(x, y)[0, 1]`	`cor(x, y)`
Eigendecomposition of a symmetric matrix	`np.linalg.eigh(S)` (ascending order)	`eigen(S, symmetric = TRUE)` (descending order)
Singular value decomposition	`np.linalg.svd(X, full_matrices=False)`	`svd(X)`
Matrix product, transpose	`A @ B`, `A.T`	`A %*% B`, `t(A)`

Task	Python	R
Norm: root of the sum of squared entries	`np.linalg.norm(X)`	`norm(X, "F")`
Clip values to $[0, 1]$	`np.clip(X, 0, 1)`	`pmin(pmax(X, 0), 1)`

Note that R counts from 1, Python from 0: the first column is `X[, 1]` in R and `X[:, 0]` in Python. In R, `var` and `cov` divide by $n - 1$; the factor $(n - 1)/n$ converts them to the divide-by-n convention this lab uses.

Exercise 1: PCA as a Variance-Maximization Problem

Suppose we have an n by p data matrix called $\mathbf{X}$ (n rows, p columns). For convenience, we have centered each of the columns of $\mathbf{X}$ around the column mean (i.e. subtracted off the column mean), so that all column means of $\mathbf{X}$ are now 0.

Our challenge is to convey as much information about $\mathbf{X}$ as we can in only one column of data (that is, with only one n-length vector). What vector would you choose?

Your first impulse might be to choose a random column from $\mathbf{X}$. It isn't much, but it preserves intact one column of $\mathbf{X}$ - and with only one column available to us, can we do any better?

The answer is yes. Suppose $\mathbf{X}$ contained one column of data that was pretty similar across observations (e.g. everyone's a Hertie student), and another that was quite different (e.g. people come to Hertie from different nations all over the world). Which column of data would you rather keep? Probably the one with *higher variance*, right? It lets you glean more information from the original matrix $\mathbf{X}$.

> An intuitive way to think of *variance* as *information* is to think in the extremes:
>
> - All observations in the dataset are students: no variation - we've learned nothing! No information!
> - All observations in the dataset are from a different country: complete variation, every observation is unique - complete information (if we had a separate database with students and their countries, we could perfectly interpolate who the students are).

Now suppose I told you that you weren't limited to one intact column of data from $\mathbf{X}$. Instead, you're allowed to take a column $\mathbf{z}_1$ that is a *linear combination* of all the columns of $\mathbf{X}$:

$$\mathbf{z}_1 = \phi_{11}\mathbf{x}_1 + \phi_{21}\mathbf{x}_2 + ... + \phi_{p1}\mathbf{x}_p = \mathbf{X}\phi_1$$

Or you can write an element of $\mathbf{z}_1$, namely z_{i1} (where $i = 1, 2, ..., n$ corresponds to the rows of the data), as

$$z_{i1} = \phi_{11}x_{i1} + \phi_{21}x_{i2} + ... + \phi_{p1}x_{ip}$$

Which linear combination would you choose? By the same logic as above, you'd pick the linear combination (i.e. the values of $\phi_{11}, \phi_{21}, ..., \phi_{p1}$) that maximizes the variance of your $\mathbf{z}_1$ vector. That is, you would solve the maximization problem

$$\begin{aligned}\max_{\phi_{11},\dots,\phi_{p1}} \text{Var}(\mathbf{z}_1) &= \max_{\phi_{11},\dots,\phi_{p1}} \frac{1}{n}\sum_{i=1}^{n}(z_{i1} - \bar{\mathbf{z}}_1)^2 \\ &= \max_{\phi_{11},\dots,\phi_{p1}} \frac{1}{n}\sum_{i=1}^{n} z_{i1}^2 \\ &= \max_{\phi_{11},\dots,\phi_{p1}} \frac{1}{n}\mathbf{z}_1^\top \mathbf{z}_1\end{aligned}$$

where the first line is the definition of variance ($\bar{\mathbf{z}}_1$ is the column mean of $\mathbf{z}_1$) and the second line follows from the fact that $\bar{\mathbf{z}}_1 = 0$ (since the means of the columns of $\mathbf{X}$ are 0). The third line is simply the vector representation of the second.

The maximal-variance $\mathbf{z}_1$ is called the **first principal component**, and we call $\phi_1 = \begin{bmatrix}\phi_{11}\\ \phi_{21}\\ \vdots \\ \phi_{p1}\end{bmatrix}$ its **loadings**.

We will constrain the loadings so that their sum of squares is equal to one, since allowing these elements to be arbitrarily large could result in an arbitrarily large variance. This adds the following constraint to our optimization problem:

$$\sum_{j=1}^{p}\phi_{j1}^2 = 1 \qquad \text{otherwise written as} \qquad \phi_1^\top\phi_1 = 1$$

Your task:

- Write down the Lagrangian for maximizing $\frac{1}{n}\mathbf{z}_1^\top\mathbf{z}_1$ subject to $\phi_1^\top\phi_1 = 1$, and express it in terms of $\mathbf{X}$ and ϕ_1.
- Take its gradient with respect to ϕ_1 and set it equal to the zero vector. Show that ϕ_1 must satisfy $\frac{1}{n}\mathbf{X}^\top\mathbf{X}\phi_1 = \lambda\phi_1$. What kind of equation is this?
- Substitute this back into $\text{Var}(\mathbf{z}_1)$. What is the variance of $\mathbf{z}_1$, and which λ should you therefore pick?

Hints.

- The transpose rule: $(A\beta)^\top = \beta^\top A^\top$.
- The gradient of a **quadratic form** $\phi^\top A\phi$ with respect to ϕ, when A is symmetric, is $\nabla_\phi(\phi^\top A\phi) = 2A\phi$. Note that $\mathbf{X}^\top\mathbf{X}$ is symmetric.
- $\phi^\top\phi$ is the quadratic form with $A = I$.

here your solution

Exercise 2: PCA as Eigendecomposition of the Variance-Covariance Matrix

The matrix $\frac{1}{n}\mathbf{X}^\top\mathbf{X}$ from Exercise 1 has a name. The **variance-covariance matrix** of a data matrix $\mathbf{X}$ is defined as

$$\mathbf{S} = \frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$$

Here $\mathbf{x}_i$ is a p-length vector of the data corresponding to row i (written as a column vector), and $\bar{\mathbf{x}}$ is a p-length vector of the *means* of the data taken over $i = 1$ to n.

Your task:

- Show that for centered data, $\mathbf{S} = \frac{1}{n}\mathbf{X}^\top\mathbf{X}$.
- Write out the (j, k) entry S_{jk}. Show that it is the covariance between columns j and k of $\mathbf{X}$, and the variance of column j when $j = k$.

> **Hint:** $\mathbf{X}^\top\mathbf{X} = \sum_{i=1}^{n}\mathbf{x}_i\mathbf{x}_i^\top$ - a sum of outer products, one per row of $\mathbf{X}$.

here your solution

So the eigenvalue equation from Exercise 1 is an eigendecomposition of the variance-covariance matrix.

> **Eigenvectors** are directions in the vector space that are invariant under the transformation defined by A. Applying A to v scales v by λ, but does not change its direction. **Eigenvalues** represent the scaling factor applied to the eigenvector.

We perform eigendecomposition on the $p \times p$ matrix $\frac{1}{n}\mathbf{X}^\top\mathbf{X}$ to get

$$\mathbf{S}\phi_1 = \lambda_1\phi_1$$

where λ_1 is the largest eigenvalue and ϕ_1 is the eigenvector corresponding to the largest eigenvalue. This eigenvector ϕ_1 is the same as the ϕ_1 that solves the constrained maximization problem from Exercise 1: it gives the loadings on the first principal component.

As before, the **first principal component** is given by

$$\mathbf{z}_1 = \phi_{11}\mathbf{x}_1 + \phi_{21}\mathbf{x}_2 + ... + \phi_{p1}\mathbf{x}_p = \mathbf{X}\phi_1$$

We can interpret this first principal component as a **projection** of our $n \times p$ matrix onto one dimension (i.e. into one n-length vector). This is the most efficient way to store our data matrix in one vector, where "most efficient" means we have maximized the variation captured in that first principal component.

It also works in terms of the matrix / vector dimensions, where we can see that $\mathbf{S}\phi_1$ "flattens" the dimensions from $p \times p$ to $p \times 1$:

- $\mathbf{S}$ is a $p \times p$ matrix,
- ϕ_1 is a $p \times 1$ vector,
- λ_1 is a scalar (scales up a matrix/vector's values).

Exercise 3: Beyond the First Principal Component

So you've chosen wisely and taken from the data matrix $\mathbf{X}$ the first principal component: the data matrix multiplied by the eigenvector corresponding to the largest eigenvalue of the variance-covariance matrix of $\mathbf{X}$. Now suppose I let you have another column, $\mathbf{z}_2$. Which column will you choose next?

If you're going the maximization route, then you should choose the linear combination of the columns of $\mathbf{X}$ that has maximal variance out of all the linear combinations that are *uncorrelated* with the first principal component $\mathbf{z}_1$. This ensures that your next column captures the most of the remaining variance in the data without duplicating information you already have in $\mathbf{z}_1$.

As you might have guessed, the answer is again an eigenvector of $\mathbf{S}$: the one with the second-largest eigenvalue. Let's check the part that makes this work - that such a column really is uncorrelated with $\mathbf{z}_1$.

Your task: let ϕ_2 be a unit eigenvector of $\mathbf{S}$ with eigenvalue $\lambda_2 \neq \lambda_1$, and $\mathbf{z}_2 = \mathbf{X}\phi_2$.

- Show that the covariance of $\mathbf{z}_1$ and $\mathbf{z}_2$ is $\phi_1^\top \mathbf{S} \phi_2$.
- Using that $\mathbf{S}$ is symmetric, compute $\phi_1^\top \mathbf{S} \phi_2$ in two ways - once with $\mathbf{S}\phi_2 = \lambda_2 \phi_2$, once with $\mathbf{S}\phi_1 = \lambda_1 \phi_1$ - and conclude that $\phi_1^\top \phi_2 = 0$.
- Conclude that $\mathbf{z}_1$ and $\mathbf{z}_2$ are uncorrelated.

Hint: the columns of $\mathbf{X}$ are centered, so every linear combination $\mathbf{z} = \mathbf{X}\phi$ has mean zero, and the covariance of $\mathbf{z}_1$ and $\mathbf{z}_2$ is $\frac{1}{n}\mathbf{z}_1^\top \mathbf{z}_2$.

here your solution

Among all the eigenvectors left, the one with the second-largest eigenvalue carries the most variance, by the same argument as in Exercise 1. We can keep constructing the next principal components from the next eigenvectors:

$$\begin{aligned} \mathbf{S}\phi_2 = \lambda_2\phi_2 &\rightarrow \mathbf{z}_2 = \mathbf{X}\phi_2 \\ \mathbf{S}\phi_3 = \lambda_3\phi_3 &\rightarrow \mathbf{z}_3 = \mathbf{X}\phi_3 \\ &\cdots \end{aligned}$$

Formally, the loading vector ϕ_k for the kth principal component is the eigenvector corresponding to the kth largest eigenvalue of $\mathbf{S}$.

Exercise 4: Enough Math, Let's Meet Laszlo the Cat

So what is our data matrix in this application? Let's do a quick tutorial on how image data is stored. We load the image of Laszlo and turn it into an **array** of three matrices:

- Each of the three matrices has 267 rows and 200 columns of data, with each data point representing a pixel of the image (our image is therefore a modest 267 by 200 pixels)
- Each of the three (267×200) matrices tells us the concentration of red, green, and blue color at that pixel
- (Any color can be represented as a combination of red, blue, and green of varying intensities.)

Load the picture of Laszlo and plot it. Explore this object. Break it down into its component `r`, `g`, and `b` matrices. Understand how these matrices are structured.

> **Tip:** Pillow reads the JPEG as integers from 0 to 255. Divide by 255 to get the 0-to-1 scale that matplotlib expects for images stored as floats.

```
# here your code
```

Exercise 5: Computing Laszlo's First Principal Component

We will now compute the first principal component of the `r` matrix, which tells us the concentration of red color in every pixel.

- Center the data (subtract the column means, but do **not** rescale).
- Check that it worked by computing the column means of the centered matrix `r_centered`.
- Compute the variance-covariance matrix of `r`. Call this object `vc`.
- Spend some time with this object. What is its dimensionality? Why does it have these dimensions?
- Reconstruct `vc[0, 0]` by hand. Interpret this value.
- Do the same for `vc[0, 1]` and `vc[0, 49]`. Interpret these values. Intuitively, why is `vc[0, 1]` a larger (more positive) value than `vc[0, 49]`?
- Now find the eigendecomposition of the variance-covariance matrix `vc`. What does it contain and how is it structured? Where are the loadings?
- Perform matrix multiplication to obtain the first principal component, $\mathbf{z}_1$. That is, multiply the centered data by the first eigenvector. Look at the dimensions of the output. Do they make sense?
- Check that the variance of $\mathbf{z}_1$ equals the largest eigenvalue λ_1.

> **Tip:** `np.cov(X, rowvar=False)` treats columns as variables, but by default it divides by $n-1$, which gives an unbiased estimate of the population covariance. For PCA the choice only rescales every eigenvalue by the same factor $n/(n-1)$, so the loadings, scores and proportions of variance are unchanged. Pass `bias=True` to divide by n, as in Exercise 1′s definition of variance, so that $\mathrm{Var}(\mathbf{z}_1) = \lambda_1$ holds exactly.

Tip: for eigendecomposition of a symmetric matrix use `np.linalg.eigh`, which returns eigenvalues in **ascending** order, so you will need to reverse them.

```
# here your code
```

here your solution

```
# here your code
```

here your solution

```
# here your code
```

here your solution

Exercise 6: The Proportion of Variance Explained

The loading vector ϕ_1 defines a direction in feature space along which the data vary the most. We can compute the **proportion of variance explained** by a principal component as the variance of that principal component divided by the total variance in the data.

The variance of the first principal component is given by

$$\frac{1}{n}\sum_{i=1}^{n} z_{i1}^2$$

And the total variance of the matrix $\mathbf{X}$ is

$$\sum_{j=1}^{p} \mathrm{Var}(\mathbf{x}_j) = \sum_{j=1}^{p} \frac{1}{n}\sum_{i=1}^{n} x_{ij}^2$$

Compute both these quantities and use them to get the total variance explained by our first principal component.

```
# here your code
```

here your solution

Exercise 7: A Reusable PCA Function

Now that we understand what's happening for the first principal component, let's write a small function to speed up computation on the remaining principal components - a bare-bones version of scikit-learn's `PCA` class.

- Write a function `pca(X, center=True)` that returns the loadings (the eigenvectors, one per column), the scores (the principal components themselves), and the standard deviation of each component.

- Compare its first eigenvector to the one you made yourself.
- Compare its first principal component to the one you made yourself.
- Check that the first and second principal components are uncorrelated, as you showed in Exercise 3.
- Check that the squared standard deviations of the components are the eigenvalues of `vc` (see the second tip below).
- Graph the proportion of variance explained for the first 5 principal components. (This is called a *scree plot*.)

Note: you can flip the sign on both an eigenvector and its associated principal component and get the same result, so you can ignore differences in signs.

Tip: rather than eigendecomposing the covariance matrix, use the **SVD** of the centered data directly - it is what scikit-learn's `PCA` does internally, and it is numerically more stable.

Why the SVD gives the same answer: write the centered data as $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, with $\mathbf{U}^\top\mathbf{U} = I$. Then

$$\mathbf{S} = \frac{1}{n}\mathbf{X}^\top\mathbf{X} = \frac{1}{n}\mathbf{V}\mathbf{\Sigma}^\top\mathbf{U}^\top\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top = \mathbf{V}\left(\frac{\mathbf{\Sigma}^\top\mathbf{\Sigma}}{n}\right)\mathbf{V}^\top$$

which is an eigendecomposition of $\mathbf{S}$: the loadings ϕ_k are the columns of $\mathbf{V}$ (the rows of `Vt`), and the eigenvalues are $\lambda_k = \sigma_k^2/n$, where σ_k are the singular values. So the standard deviation of the k-th component is $\sqrt{\lambda_k} = \sigma_k/\sqrt{n}$.

```
def pca(X, center=True):
   """PCA via the SVD of the (centered) data: returns (loadings, scores, sdev)."""
    # center the columns if `center` is True
    # here your code
    # run the SVD on the (centered) matrix
    # here your code
    # loadings: p x k, the columns are the eigenvectors
    # scores:   n x k, the principal components
    # sdev:     the standard deviation of each component (divide by n)
    # here your code
    return loadings, scores, sdev


r_load, r_scores, r_sdev = pca(r, center=True)
```

```
print("loadings shape:", r_load.shape)
print("scores shape:  ", r_scores.shape)
```

```
# here your code
```

```
# scree plot: proportion of variance explained by the first 5 components
def scree_plot(sdev, ax, title, color, ncp=5):
    pve = sdev ** 2 / np.sum(sdev ** 2) * 100
    ax.bar(np.arange(1, ncp + 1), pve[:ncp], color=color, alpha=0.7)
    for i, v in enumerate(pve[:ncp]):
        ax.text(i + 1, v + 1, f"{v:.1f}%", ha="center", fontsize=8)
    ax.set_xlabel("Principal component")
    ax.set_ylabel("% variance explained")
    ax.set_title(title)
    ax.set_ylim(0, min(100, pve[:ncp].max() * 1.25))

# here your code
#
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
scree_plot(r_sdev, axes[0], "Red", "red")
scree_plot(g_sdev, axes[1], "Green", "green")
scree_plot(b_sdev, axes[2], "Blue", "blue")
fig.tight_layout()
plt.show()
```

Exercise 8: Using PCA for Image Compression

Let's see what this image actually looks like for different numbers of principal components. Start by running PCA on the centered `r`, `g`, and `b` matrices, and keep the column means of each channel: you will need them to turn a reconstruction back into an image.

```
# here your code
```

To see what image is created by the first principal component, project $\mathbf{z}_1$ back into p-dimensional space by multiplying it by $\phi_1^\top$. This rebuilds the *centered* data, so add the column means back before you plot the image.

How $A^\top$ reverses a transformation - illustrated with dimensions.

Forward transformation (compression): $\mathbf{z}_1 = \mathbf{X}\phi_1$

- $\mathbf{X}$: $(n \times p)$ - original data

- ϕ_1: $(p \times 1)$ - first eigenvector / loading vector
- $\mathbf{z}_1$: $(n \times 1)$ - first principal component (compressed to 1 dimension)

Reverse transformation (reconstruction): $\hat{\mathbf{X}} = \mathbf{z}_1 \phi_1^\top$

- $\mathbf{z}_1$: $(n \times 1)$ - compressed data
- $\phi_1^\top$: $(1 \times p)$ - transposed loading vector
- $\hat{\mathbf{X}}$: $(n \times p)$ - reconstructed data (back to original dimensions!)

The transpose operation effectively "reverses" the dimensional reduction: multiplying by ϕ_1 projects from p dimensions down to 1, while multiplying by $\phi_1^\top$ projects from 1 dimension back up to p dimensions. Though we lose information (we can't perfectly reconstruct), we recover the projection onto that principal component's direction in the original feature space.

```
def reconstruct(rgb_pca, rgb_means, k):
   """Rebuild the image from the first k principal components of each channel."""
    channels = []
    for (loadings, scores, _), means in zip(rgb_pca, rgb_means):
        # recompose the compressed (centered) image: (n x k) @ (k x p) = (n x p)
        # call it new_rgb
        # here your code
        # add the column means back to undo the centering
        # here your code
          # clip, don't rescale: values that overshoot [0, 1] become blown-
out pixels

        channels.append(np.clip(new_rgb, 0, 1))
    return np.stack(channels, axis=2)


expanded_1 = reconstruct(rgb_pca, rgb_means, 1)

fig, ax = plt.subplots(figsize=(3.5, 4.5))
ax.imshow(expanded_1)
ax.set_title("1 principal component")
ax.axis("off")
plt.show()
```

Although you don't see a cat, this is already pretty impressive: you get the general color palette of the foreground and background of the picture. Repeat this by looping over some values of principal components and plot the results. **How many principal components does it take to recognize Laszlo the cat?**

```
# here your code
```

here your solution

What does compression cost? Take the red channel and its reconstruction $\hat{\mathbf{X}}$ from the first $k = 20$ components (before clipping). Compute the mean squared reconstruction error per row,

$$\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{p}(x_{ij} - \hat{x}_{ij})^2$$

and compare it to the sum of the eigenvalues you dropped, $\lambda_{21} + \ldots + \lambda_{200}$. What do you notice?

```
k = 20
# calculate the error and dropped
# here your code
#
print(f"reconstruction error:  {error:.6f}")
print(f"dropped eigenvalues:   {dropped:.6f}")
print("equal?", np.isclose(error, dropped))
```

here your solution

Exercise 9: A Black and White Image

[optional]

Try PCA on a black and white image. Create a new matrix that is the sum of the `r`, `g`, and `b` matrices, then divide every cell by the maximum value. Repeat PCA with this object.

Does the black and white image need *fewer* components than a single color channel? Judging by eye is unreliable, so measure it: for $k = 1, 5, 10, 20$, compute the **relative reconstruction error**

$$\frac{\left\|\mathbf{X} - \hat{\mathbf{X}}\right\|}{\|\mathbf{X}\|}$$

for `bw` and for each of `r`, `g` and `b`, where $\hat{\mathbf{X}}$ is the reconstruction from the first k components (with the column means added back, before clipping) and $\|\cdot\|$ is the square root of the sum of all squared entries (`np.linalg.norm`).

```
# here your code
```

here your solution